# 🧠 Practice Set — System Building Tasks (Level 2)

**Note: Only for trainees who started from 8th February 2026.**

---

## ✅ Task 1 — Student Record Manager

### What You're Building

A system that can store **up to 4 students** with:

1. Student Name (TEXT)
2. Age (NUMBER)
3. Marks (NUMBER)
4. Status (TEXT → Pass / Fail)

---

### Menu Options

1. Add Student
2. View Students
3. Update Student Marks
4. Remove Student
5. Exit

Menu must repeat until Exit.

---

### Feature Rules

### Add Student

- Maximum 4 students only
- Status rules:
  - Marks ≥ 50 → Pass
  - Otherwise → Fail

If full → show error message.

---

### View Students

Display:

NAME | AGE | MARKS | STATUS

Also show:

- Total Students
- Average Marks

---

**Update Student Marks**

- Ask which student position (1–4)
- Show current data
- Enter new marks
- Recalculate status

---

**Remove Student**

Use **mark as deleted approach**

---

🎯 **Concepts Practiced**

- Derived values (status from marks)
- Average calculation
- Data updates

---

✅ **Task 2 — Library Book Tracker**

**Data Per Book (Max 3)**

1. Book Title
2. Author
3. Is Borrowed (BOOLEAN)

---

**Menu**

1. Add Book
2. Borrow Book
3. Return Book
4. View Books
5. Exit

---

**Rules**

Borrow Book:

- Only if not already borrowed
- Otherwise show:
  "Book already borrowed"

Return Book:

- Only if currently borrowed

View:

Show:

TITLE | AUTHOR | AVAILABLE / BORROWED

Also show:

- Total books
- Borrowed count

---

### 🎯 Concepts Practiced

- Boolean state changes
- Counters
- Validation logic

---

### ✅ Task 3 — Simple Bank Account System

Max 3 accounts.

Each account:

1. Account Name
2. Balance

---

**Menu**

1. Create Account
2. Deposit
3. Withdraw

4. View Accounts
5. Exit

---

**Rules**

Withdraw:

- Cannot withdraw more than balance
- Cannot withdraw ≤ 0

Deposit:

- Amount must be > 0

Display warning if balance becomes 0:

"Account Empty"

---

🎯 **Concepts Practiced**

- Arithmetic updates
- Validation ordering
- Financial logic

---

✅ **Task 4 — Employee Attendance System**

Max 5 employees.

Data:

1. Employee Name
2. Days Present (NUMBER)
3. Days Absent (NUMBER)

---

**Menu**

1. Add Employee
2. Record Attendance
3. View Report
4. Remove Employee
5. Exit

---

**Rules**

Attendance Percentage:

PERCENT = PRESENT / (PRESENT + ABSENT) * 100

Status:

- ≥ 75 → Good
- Otherwise → Warning

Report shows:

NAME | PRESENT | ABSENT | % | STATUS

---

### 🎯 Concepts Practiced

- Formulas
- Percentage
- Derived status

---

### ✅ Task 5 — Product Inventory System

Max 4 products.

Data:

1. Product Name
2. Price
3. Quantity

---

**Menu**

1. Add Product
2. Sell Product
3. Restock Product
4. View Inventory
5. Exit

---

**Rules**

Sell:

- Quantity must be available
- If quantity becomes 0 → show:

"Out of Stock"

Restock:

- Add quantity to existing stock

View:

Show:

NAME | PRICE | QTY | TOTAL VALUE

Where:

TOTAL VALUE = PRICE * QTY

Also show:

Total Inventory Value.

---

🎯 **Concepts Practiced**

- Multiplication
- Stock management
- Accumulation totals

---

✅ **Task 6 — Exam Attempt System (Logic Heavy)**

System allows **max 3 attempts per student**.

Menu:

1. Attempt Exam
2. View Result
3. Reset Attempts
4. Exit

---

Rules:

Pass if score ≥ 60.

If failed 3 times → show:

"Locked — Contact Admin"

---

### 🎯 Concepts Practiced

- Attempt counters
- Limits
- Lock conditions

---

### ⭐ Challenge Task (Advanced Thinking)

**Parking Slot Manager**

You have **5 parking slots**.

Each slot stores:

- Car Number
- Is Occupied (BOOLEAN)

Menu:

1. Park Car
2. Remove Car
3. View Parking
4. Exit

Rules:

- Cannot park if full
- Cannot remove if empty
- Show available slot count

Explanation:

### ✅ Task 1 — Student Record Manager (Beginner Explanation)

Imagine you are a teacher who has a small notebook that can hold information for only **4 students**.

For each student, you want to write:

- Their name
- Their age
- Their marks
- Whether they passed or failed

The computer program will act like this notebook.

The system will show you a menu again and again asking what you want to do, such as:

- Add a new student
- See all students
- Update marks
- Remove a student
- Exit

When adding a student, the program must check if there is space available (because you can only store 4). If the notebook is full, it should tell you that no more students can be added.

The program should automatically decide whether the student passed or failed based on their marks. For example, if marks are 50 or more, they pass.

When viewing students, the system should also calculate the **average marks**, which means the total marks divided by number of students.

This task helps you understand how to store and update information.

---

### ✅ Task 2 — Library Book Tracker

Imagine you manage a very small library with space for only **3 books**.

For each book, you want to store:

- Book title
- Author name
- Whether the book is borrowed or available

The program menu will let you:

- Add a book

- Borrow a book
- Return a book
- View all books
- Exit

If someone tries to borrow a book that is already borrowed, the program should warn them.

If someone tries to return a book that was never borrowed, the program should also warn them.

When viewing books, the program should clearly show whether each book is available or borrowed.

This task teaches you how to track **status using TRUE or FALSE** values.

---

### ✅ Task 3 — Simple Bank Account System

Think of a small bank that can manage only **3 accounts**.

Each account has:

- Account holder name
- Balance (money in account)

The system allows you to:

- Create an account
- Deposit money
- Withdraw money
- View accounts
- Exit

The program must check rules carefully:

- You cannot withdraw more money than you have.
- You cannot deposit zero or negative money.
- If the balance becomes zero, the system should show a message like “Account Empty”.

This task teaches you how numbers change over time using calculations.

---

### ✅ Task 4 — Employee Attendance System

Imagine you run a small office with **5 employees**.

For each employee, you store:

- Name

- Days present
- Days absent

The system lets you:

- Add employees
- Record attendance
- View report
- Remove employee
- Exit

The interesting part is calculating **attendance percentage**.

For example:

If someone worked 15 days and was absent 5 days:

Total days = 20
Percentage = 15 / 20 × 100 = 75%

Then the program decides:

- 75% or more → Good
- Less → Warning

This task teaches formulas and percentages.

---

### ✅ Task 5 — Product Inventory System

Imagine you own a small shop with **4 products**.

For each product, you store:

- Product name
- Price
- Quantity available

The system allows:

- Add product
- Sell product
- Restock product
- View inventory
- Exit

If someone tries to sell more items than available, the program must stop them.

If quantity becomes zero, the system should show “Out of Stock”.

When viewing inventory, the program should also calculate the **total value** of each product:

Price × Quantity

And also calculate total value of the entire shop stock.

This task teaches multiplication and stock management.

---

### ✅ Task 6 — Exam Attempt System

Imagine a student is allowed to take an exam only **3 times**.

The system allows:

- Attempt exam
- View result
- Reset attempts
- Exit

Rules:

- Score ≥ 60 → Pass
- Score < 60 → Fail
- If student fails 3 times → system locks them

This teaches counters and limits.

---

### ⭐ Challenge Task — Parking Slot Manager

Imagine a parking area with **5 parking spaces**.

Each space can either be:

- Empty
- Occupied by a car

The system allows:

- Park a car
- Remove a car
- View parking status
- Exit

If parking is full, no more cars can enter.

If someone tries to remove a car from an empty slot, the system should show an error.

This teaches space management and tracking availability.

---

### 🧠 What All These Tasks Are Teaching (Big Picture)

Without realizing it, you are learning the core of programming:

- Storing information
- Making decisions
- Repeating actions
- Counting items
- Calculating values
- Managing data
- Handling errors

These are the **same skills used in real software companies**.